HW3: Predictive Control of Dynamic System (ME 675)

Instructor: Youngki Kim

Due Date: Nov 2nd via Canvas

Problem 1 Consider a rotational dynamics model given by:

$$\ddot{\theta} = \frac{u}{J}$$

where θ is the angle of rotation, u is the control. Let the inertia J be one.

(1) Express the model in state-space representation.

(2) Assuming a sampling period of 0.25 sec, convert the continuous-time state equations to discrete time state update equations.

(3) Consider the following minimum control effort optimal control problem for the system in (2).

$$\min \quad J = \sum_{k=0}^{N-1} u_k^T u_k$$

where $N = 20$ subject to a terminal constraint, $\theta_N = \dot{\theta}_N = 0$. Solve this problem using `fmincon.m`, by treating the inputs $u_0, u_1, \ldots, u_{N-1}$ as optimization variables. Plot the closed-loop trajectory on the phase plane for the initial condition $(\frac{\pi}{2}, 0)$. Note: take this as a nonlinear programming.

(4) Consider the solution of the problem in (3) based on necessary conditions for optimality. Write out the transversality conditions. Using the state transition matrix approach for states and adjoint variables, determine the optimal control sequence. Compare your result with that in (3).

(5) Solve the problem via P-LQN formulation and compare the result with that in (3). Note: use `quadprog.m` without terminal constraints.

```
clear all
% States
syms theta theta_dot theta_dot_dot J u
theta
```

theta = θ

```
theta_dot
```

theta_dot = $\dot{\theta}$

```
theta_dot_dot
```

theta_dot_dot = $\ddot{\theta}$

```
StateVector = [theta; theta_dot]
```

```
StateVector =
```

$$\begin{pmatrix} \theta \\ \dot{\theta} \end{pmatrix}$$

Problem 1.1: System in CT State Space

```
% Define the continuous-time state-space representation
A = [0 1; 0 0];
B = [0; 1]; %for J=1
C = eye(2);
D = zeros(2, 1);
Ts = 0.25
```

```
Ts =
0.2500
```

```
LTI_sys = ss(A,B,C,D)
```

```
LTI_sys =

  A =
       x1  x2
   x1   0   1
   x2   0   0

  B =
       u1
   x1   0
   x2   1

  C =
       x1  x2
   y1   1   0
   y2   0   1

  D =
       u1
   y1   0
   y2   0

Continuous-time state-space model.
Model Properties
```

```
x0 = [pi/2; 0] %IC
```

```
x0 = 2×1
    1.5708
         0
```

```
xN = [0; 0];   %TC
```

Problem 1.2: System in DT State Space

```
LTI_sys_d = c2d(LTI_sys,Ts) %Discrete System
```

```
LTI_sys_d =

  A =
```

```
          x1    x2
   x1      1  0.25
   x2      0     1

  B =
             u1
   x1  0.03125
   x2     0.25

  C =
       x1  x2
   y1   1   0
   y2   0   1

  D =
       u1
   y1   0
   y2   0

Sample time: 0.25 seconds
Discrete-time state-space model.
Model Properties
```

```
Ad = LTI_sys_d.A
```

```
Ad = 2×2
    1.0000    0.2500
         0    1.0000
```

```
Bd = LTI_sys_d.B
```

```
Bd = 2×1
    0.0312
    0.2500
```

```
Cd = LTI_sys_d.C
```

```
Cd = 2×2
     1     0
     0     1
```

```
Dd = LTI_sys_d.D
```

```
Dd = 2×1
     0
     0
```

Problem 1.3: Closed Loop MPC Formulation

```
%Cost Function f_cost
% f_cost = @(u) sum(u.^2)
Ncost=20; %optimizes cost over 20 horizon
Nterminal = 20; %optimization constraint, Nterminal = i, x must reach xN by i on
the horizon
U_guess = rand(Ncost,1);

solver_options = optimoptions('fmincon', ...
    'Display','iter', ...
    'Algorithm','sqp', ...
```

```
    'MaxFunctionEvaluations',5e4, ...
    'TolFun',1e-9, ...
    'TolCon',1e-9);

function J = f_cost(U,Ncost)
% U is a vertically concatenated vector of control input vectors u.
% Our system is single input so ui is 1x1
% For L input system & N horizon, U has dimension LN x 1. Function
% returns cost for the series of inputs over the horizon (applies u1,
% u2, ..., uN)

J = 0;
    for k=0:Ncost-1
        u_inc = U(k+1);
        J_inc = sum(u_inc.^2);
        J = J + J_inc;
    end
end

function [c, ceq] = nonlcon(U, Ncost, Nterminal, x0, xN, Ad, Bd)
% c <= 0 & ceq == 0 are given to fmincon as constraints on the optimization
% where c & ceq are nonlinear functions of U, the optimization vector (stack of
inputs)
% x_prediction moves along horizon with system dynamics to N where
% terminal constraint x(:,N) = xN defined above is set using ceq.
% This constraint is linear so it should probably not be under nonlcon
% fmincon input, but oh well

    x_prediction = [x0 zeros(2,Ncost-1)];
    for k=0:Ncost-1
        u_inc = U(k+1);
        x_prediction(:,k+2) = Ad*x_prediction(:,k+1) + Bd*u_inc;
    end

% Terminal Constraint: x_prediction(:,N) = x0, solver takes ceq = 0 as
% constraint:

%the code below until "%ends_here" is not really used, I just wanted to see
%what would happen if I moved the terminal constraint closer and closer w/
%each optimization, to show x would reach xN in exactly N steps.
number_of_xN_on_horizon = Ncost+1 - Nterminal;
vector_of_xN = zeros(2,number_of_xN_on_horizon);

% vector_of_xN = [xN, xN, xN, ... xN] with number_of_xN_on_horizon columns
for i=1:number_of_xN_on_horizon
    vector_of_xN(:,i) = xN;
end

%ends_here
ceq = x_prediction(:,Nterminal:Ncost)-vector_of_xN;
```

```
c=[];
end

%Anonymous cost and constraint function handles so that the solver can run
%different values of U, and the other arguments are grabbed from
%earlier in the script. Lets find U_optimal at x0 and plot
f_cost_anonymous = @(U) f_cost(U,Ncost)
```

```
f_cost_anonymous = function_handle with value:
    @(U)f_cost(U,Ncost)
```

```
nonlcon_anonymous = @(U) nonlcon(U, Ncost, Nterminal, x0, xN, Ad, Bd)
```

```
nonlcon_anonymous = function_handle with value:
    @(U)nonlcon(U,Ncost,Nterminal,x0,xN,Ad,Bd)
```

```
U_optimal1 = fmincon(f_cost_anonymous,U_guess,[],[],[],[],[],[],nonlcon_anonymous)
```

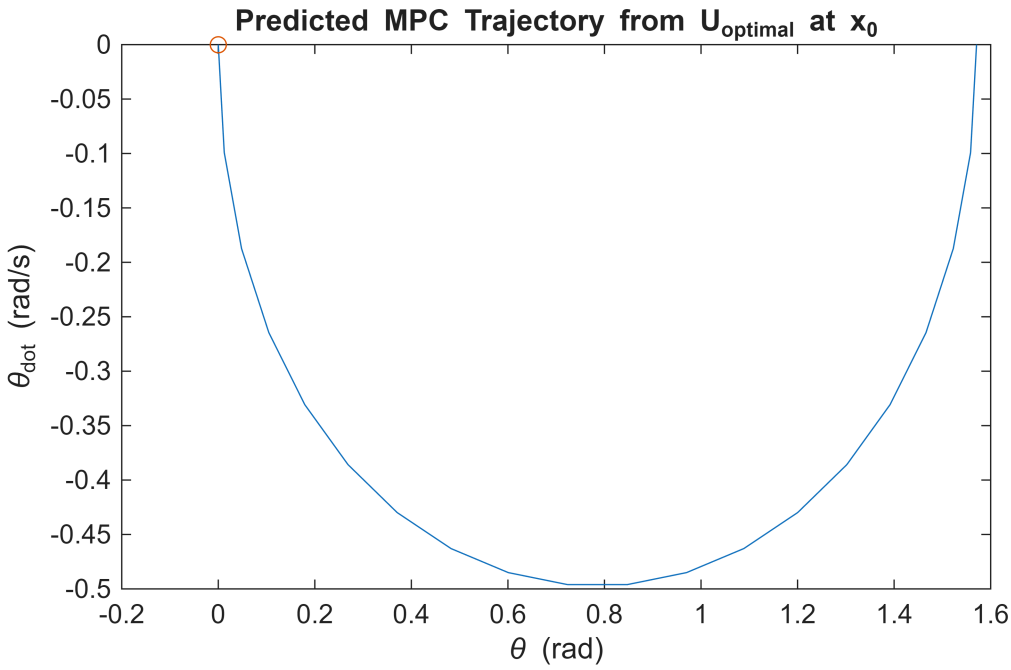
```
Local minimum found that satisfies the constraints.

Optimization completed because the objective function is non-decreasing in
feasible directions, to within the value of the optimality tolerance,
and constraints are satisfied to within the value of the constraint tolerance.

<stopping criteria details>
U_optimal1 = 20×1
   -0.3968
   -0.3527
   -0.3086
   -0.2646
   -0.2205
   -0.1764
   -0.1323
   -0.0882
   -0.0441
    0.0000
    0.0441
    0.0882
    0.1323
    0.1764
    0.2205
      ⋮
```

```
x_disc = [x0, zeros(2,Ncost-1)]
```

```
x_disc = 2×20
    1.5708         0         0         0         0         0         0         0 ···
         0         0         0         0         0         0         0         0
```

```
for k=1:Ncost-1
x_disc(:,k+1) = Ad*x_disc(:,k) + Bd*U_optimal1(k);
end

plot(x_disc(1,:),x_disc(2,:))
hold on
scatter(x_disc(1,Ncost), x_disc(2,Ncost),'Color','r')
xlabel('\theta (rad)'); ylabel('\theta_d_o_t (rad/s)');
```

```
title('Predicted MPC Trajectory from U_o_p_t_i_m_a_l at x_0');
```




```
figure;
```

Problem 1.3: Closed Loop MPC Trajectory

```
%Close the loop and run for 30 time steps to see if the trajectory
%converges to xN. Input comes from the nonlinear LQN solution U, the
%solution to the quadratic program with terminal constraint.
%(first u in U_optimal at each optimization)

x_cl = [x0 zeros(2,Ncost)];
x_next = x0;
loops = 20;
U_optimal = zeros(Ncost,loops);
u_applied = zeros(1,loops);
for k = 1:loops

    % Define the constraint with current state
    nonlcon_current = @(U) nonlcon(U, Ncost, Nterminal, x_next, xN, Ad, Bd);
    %Nterminal = Ncost-k;    %If I keep bringing the terminal state closer
    %on the horizon, convergence to xN will happen in N steps. But that
    %defeats the purpose. we want the moving horizon, thats the point of
    %MPC!!!!

    % Solve MPC problem from current state
    U_optimal(:,k) = fmincon(f_cost_anonymous, U_guess, [],[],[],[],[],[],
nonlcon_current, solver_options);

    % Apply first control input
    u_applied(:,k) = U_optimal(1,k);
```

```
    % Update system state
    x_next = Ad*x_next + Bd*u_applied(:,k);

    % Store closed-loop trajectory
    x_cl(:,k+1) = x_next;

    % Optionally update initial guess for next optimization
    % U_guess = [U_optimal(2:end); U_optimal(end)];
    U_guess = rand(Ncost,1)
end
```

```
 Iter  Func-count            Fval   Feasibility   Step Length       Norm of   First-order
                                                                       step    optimality
    0          21    4.579345e+00     6.874e+00     1.000e+00     0.000e+00     1.902e+00
    1          42    2.462113e+00     4.444e-15     1.000e+00     3.335e+00     1.585e+00
    2          63    1.269262e+00     2.082e-16     1.000e+00     1.565e+00     4.550e-01
    3          84    1.108166e+00     4.094e-16     1.000e+00     4.014e-01     4.470e-08
    4          85    1.108166e+00     4.094e-16     7.000e-01     2.237e-16     4.470e-08

Local minimum possible. Constraints satisfied.

fmincon stopped because the size of the current step is less than
the value of the step size tolerance and constraints are
satisfied to within the value of the constraint tolerance.

<stopping criteria details>
U_guess = 20×1
    0.8593
    0.9742
    0.5708
    0.9969
    0.5535
    0.5155
    0.3307
    0.4300
    0.4918
    0.0710
    0.8877
    0.0646
    0.4362
    0.8266
    0.3945
      ⋮
 Iter  Func-count            Fval   Feasibility   Step Length       Norm of   First-order
                                                                       step    optimality
    0          21    8.677178e+00     8.018e+00     1.000e+00     0.000e+00     1.994e+00
    1          42    2.324350e+00     3.013e-15     1.000e+00     3.686e+00     1.609e+00
    2          63    1.013841e+00     1.527e-16     1.000e+00     1.706e+00     5.949e-01
    3          84    7.941345e-01     5.031e-17     1.000e+00     4.687e-01     5.031e-08
    4          85    7.941345e-01     5.031e-17     7.000e-01     3.022e-08     4.818e-08

Local minimum possible. Constraints satisfied.

fmincon stopped because the size of the current step is less than
the value of the step size tolerance and constraints are
satisfied to within the value of the constraint tolerance.

<stopping criteria details>
U_guess = 20×1
```

```
    0.2586
    0.8979
    0.5934
    0.5038
    0.6128
    0.8194
    0.5319
    0.2021
    0.4539
    0.4279
    0.9661
    0.6201
    0.6954
    0.7202
    0.3469
      ⋮

 Iter  Func-count            Fval   Feasibility   Step Length       Norm of   First-order
                                                                       step    optimality
    0          21    7.079421e+00     7.085e+00     1.000e+00     0.000e+00     1.932e+00
    1          42    1.893645e+00     2.068e-15     1.000e+00     3.322e+00     1.410e+00
    2          63    7.361832e-01     8.448e-16     1.000e+00     1.555e+00     5.141e-01
    3          84    5.721237e-01     8.327e-17     1.000e+00     4.050e-01     3.029e-08
    4          85    5.721237e-01     8.327e-17     7.000e-01     1.269e-08     2.980e-08

Local minimum possible. Constraints satisfied.

fmincon stopped because the size of the current step is less than
the value of the step size tolerance and constraints are
satisfied to within the value of the constraint tolerance.

<stopping criteria details>
U_guess = 20×1
    0.4265
    0.8363
    0.7314
    0.3600
    0.4542
    0.3864
    0.7756
    0.7343
    0.4303
    0.6938
    0.9452
    0.7842
    0.7056
    0.1093
    0.3899
      ⋮

 Iter  Func-count            Fval   Feasibility   Step Length       Norm of   First-order
                                                                       step    optimality
    0          21    7.189621e+00     6.935e+00     1.000e+00     0.000e+00     1.890e+00
    1          42    2.084101e+00     1.995e-15     1.000e+00     3.477e+00     1.668e+00
    2          63    5.563515e-01     2.914e-16     1.000e+00     1.664e+00     4.821e-01
    3          84    4.171858e-01     4.857e-16     1.000e+00     3.730e-01     4.541e-08
    4         105    4.171858e-01     4.666e-16     1.000e+00     3.678e-16     4.541e-08

Local minimum possible. Constraints satisfied.

fmincon stopped because the size of the current step is less than
the value of the step size tolerance and constraints are
satisfied to within the value of the constraint tolerance.
```

```
<stopping criteria details>
U_guess = 20×1
    0.0156
    0.8637
    0.0781
    0.6690
    0.5002
    0.2180
    0.5716
    0.1222
    0.6712
    0.5996
    0.0560
    0.0563
    0.1525
    0.0196
    0.4352
      ⋮
 Iter  Func-count            Fval   Feasibility   Step Length       Norm of   First-order
                                                                       step    optimality
    0          21    4.968766e+00     4.326e+00     1.000e+00     0.000e+00     1.728e+00
    1          42    1.951020e+00     4.840e-16     1.000e+00     2.999e+00     1.511e+00
    2          63    3.504932e-01     4.302e-16     1.000e+00     1.481e+00     2.768e-01
    3          84    3.103490e-01     4.536e-16     1.000e+00     2.004e-01     6.239e-08
    4          85    3.103490e-01     4.536e-16     7.000e-01     4.177e-08     6.239e-08

Local minimum possible. Constraints satisfied.

fmincon stopped because the size of the current step is less than
the value of the step size tolerance and constraints are
satisfied to within the value of the constraint tolerance.

<stopping criteria details>
U_guess = 20×1
    0.9081
    0.1080
    0.5170
    0.1432
    0.5594
    0.0046
    0.7667
    0.8487
    0.9168
    0.9870
    0.5051
    0.2714
    0.1008
    0.5078
    0.5856
      ⋮
 Iter  Func-count            Fval   Feasibility   Step Length       Norm of   First-order
                                                                       step    optimality
    0          21    6.822640e+00     5.621e+00     1.000e+00     0.000e+00     1.974e+00
    1          42    2.052035e+00     3.149e-15     1.000e+00     3.348e+00     1.394e+00
    2          63    3.203996e-01     5.681e-16     1.000e+00     1.635e+00     4.183e-01
    3          84    2.373960e-01     2.359e-16     1.000e+00     2.881e-01     4.470e-08
    4          85    2.373960e-01     2.359e-16     7.000e-01     2.862e-08     4.470e-08

Local minimum possible. Constraints satisfied.

fmincon stopped because the size of the current step is less than
```

```
the value of the step size tolerance and constraints are
satisfied to within the value of the constraint tolerance.

<stopping criteria details>
U_guess = 20×1
    0.9386
    0.5905
    0.4406
    0.9419
    0.6559
    0.4519
    0.8397
    0.5326
    0.5539
    0.6801
    0.3672
    0.2393
    0.5789
    0.8669
    0.4068
      ⋮
 Iter  Func-count            Fval   Feasibility   Step Length       Norm of   First-order
                                                                       step    optimality
    0          21    7.303068e+00     6.631e+00     1.000e+00     0.000e+00     1.884e+00
    1          42    1.552603e+00     1.347e-15     1.000e+00     3.235e+00     1.667e+00
    2          63    3.228003e-01     4.770e-17     1.000e+00     1.536e+00     5.241e-01
    3          84    1.878410e-01     1.596e-16     1.000e+00     3.674e-01     3.637e-08
    4          85    1.878410e-01     1.596e-16     7.000e-01     1.974e-08     3.637e-08

Local minimum possible. Constraints satisfied.

fmincon stopped because the size of the current step is less than
the value of the step size tolerance and constraints are
satisfied to within the value of the constraint tolerance.

<stopping criteria details>
U_guess = 20×1
    0.4036
    0.3902
    0.3604
    0.1403
    0.2601
    0.0868
    0.4294
    0.2573
    0.2976
    0.4249
    0.1192
    0.4951
    0.7064
    0.2436
    0.7851
      ⋮
 Iter  Func-count            Fval   Feasibility   Step Length       Norm of   First-order
                                                                       step    optimality
    0          21    2.702894e+00     3.173e+00     1.000e+00     0.000e+00     1.570e+00
    1          42    9.032634e-01     1.246e-15     1.000e+00     2.056e+00     1.133e+00
    2          63    1.758310e-01     3.443e-16     1.000e+00     1.013e+00     2.013e-01
    3          84    1.540850e-01     1.414e-16     1.000e+00     1.475e-01     5.029e-08
    4         105    1.540850e-01     1.414e-16     1.000e+00     5.075e-17     5.029e-08

Local minimum possible. Constraints satisfied.
```

```
fmincon stopped because the size of the current step is less than
the value of the step size tolerance and constraints are
satisfied to within the value of the constraint tolerance.

<stopping criteria details>
U_guess = 20×1
    0.1892
    0.1425
    0.2681
    0.1749
    0.1386
    0.5989
    0.9011
    0.9394
    0.2212
    0.4827
    0.3760
    0.5238
    0.2649
    0.0684
    0.4363
      ⋮
 Iter  Func-count            Fval   Feasibility   Step Length       Norm of   First-order
                                                                       step    optimality
    0          21    5.245931e+00     3.526e+00     1.000e+00     0.000e+00     1.923e+00
    1          42    2.538739e+00     5.326e-16     1.000e+00     3.387e+00     1.923e+00
    2          63    1.490433e-01     5.161e-16     1.000e+00     1.687e+00     2.113e-01
    3          84    1.307296e-01     2.567e-16     1.000e+00     1.353e-01     4.778e-08
    4          85    1.307296e-01     2.567e-16     7.000e-01     2.283e-08     4.282e-08

Local minimum possible. Constraints satisfied.

fmincon stopped because the size of the current step is less than
the value of the step size tolerance and constraints are
satisfied to within the value of the constraint tolerance.

<stopping criteria details>
U_guess = 20×1
    0.7624
    0.0073
    0.6800
    0.7060
    0.6451
    0.5523
    0.2181
    0.7724
    0.2280
    0.3709
    0.8909
    0.8564
    0.4024
    0.3180
    0.6086
      ⋮
 Iter  Func-count            Fval   Feasibility   Step Length       Norm of   First-order
                                                                       step    optimality
    0          21    8.100926e+00     5.491e+00     1.000e+00     0.000e+00     1.820e+00
    1          42    2.091998e+00     1.587e-15     1.000e+00     3.541e+00     1.706e+00
    2          63    2.153333e-01     4.389e-16     1.000e+00     1.725e+00     4.127e-01
    3          84    1.140263e-01     2.177e-16     1.000e+00     3.183e-01     5.202e-08
    4          85    1.140263e-01     2.177e-16     7.000e-01     2.755e-08     4.399e-08
```

```
Local minimum possible. Constraints satisfied.

fmincon stopped because the size of the current step is less than
the value of the step size tolerance and constraints are
satisfied to within the value of the constraint tolerance.

<stopping criteria details>
U_guess = 20×1
    0.4424
    0.9044
    0.0332
    0.5324
    0.7165
    0.1793
    0.3365
    0.1877
    0.3219
    0.4039
    0.5486
    0.0487
    0.5527
    0.2748
    0.2415
      ⋮
 Iter  Func-count            Fval   Feasibility   Step Length       Norm of   First-order
                                                                       step    optimality
    0          21    5.544929e+00     3.893e+00     1.000e+00     0.000e+00     1.913e+00
    1          42    2.254619e+00     4.718e-16     1.000e+00     3.309e+00     1.637e+00
    2          63    1.321073e-01     1.665e-16     1.000e+00     1.642e+00     2.470e-01
    3          84    1.014434e-01     2.125e-17     1.000e+00     1.751e-01     1.770e-08
    4          85    1.014434e-01     2.125e-17     7.000e-01     1.669e-08     1.770e-08

Local minimum possible. Constraints satisfied.

fmincon stopped because the size of the current step is less than
the value of the step size tolerance and constraints are
satisfied to within the value of the constraint tolerance.

<stopping criteria details>
U_guess = 20×1
    0.7283
    0.1758
    0.3604
    0.1888
    0.0012
    0.3164
    0.6996
    0.6253
    0.5431
    0.4390
    0.2874
    0.5017
    0.7615
    0.7624
    0.5761
      ⋮
 Iter  Func-count            Fval   Feasibility   Step Length       Norm of   First-order
                                                                       step    optimality
    0          21    5.388225e+00     4.150e+00     1.000e+00     0.000e+00     1.525e+00
    1          42    1.135662e+00     1.638e-15     1.000e+00     2.712e+00     1.040e+00
    2          63    1.705897e-01     3.764e-16     1.000e+00     1.303e+00     4.285e-01
```

```
    3          84    9.132714e-02     7.112e-17     1.000e+00     2.815e-01     2.323e-08
    4         105    9.132714e-02     3.990e-17     1.000e+00     2.857e-17     1.717e-08

Local minimum possible. Constraints satisfied.

fmincon stopped because the size of the current step is less than
the value of the step size tolerance and constraints are
satisfied to within the value of the constraint tolerance.

<stopping criteria details>
U_guess = 20×1
    0.0921
    0.1478
    0.1982
    0.6723
    0.4315
    0.6944
    0.2568
    0.0098
    0.5323
    0.2794
    0.9462
    0.9064
    0.3927
    0.0249
    0.6714
      ⋮
 Iter  Func-count            Fval   Feasibility   Step Length       Norm of   First-order
                                                                       step    optimality
    0          21    6.129995e+00     3.717e+00     1.000e+00     0.000e+00     1.943e+00
    1          42    2.168497e+00     4.753e-16     1.000e+00     3.368e+00     1.358e+00
    2          63    1.309910e-01     2.671e-16     1.000e+00     1.664e+00     3.860e-01
    3          84    8.264447e-02     4.510e-16     1.000e+00     2.199e-01     4.325e-08
    4         105    8.264447e-02     8.066e-17     1.000e+00     3.426e-08     7.413e-08

Local minimum possible. Constraints satisfied.

fmincon stopped because the size of the current step is less than
the value of the step size tolerance and constraints are
satisfied to within the value of the constraint tolerance.

<stopping criteria details>
U_guess = 20×1
    0.6866
    0.7194
    0.6500
    0.7269
    0.3738
    0.5816
    0.1161
    0.0577
    0.9798
    0.2848
    0.5950
    0.9622
    0.1858
    0.1930
    0.3416
      ⋮
 Iter  Func-count            Fval   Feasibility   Step Length       Norm of   First-order
                                                                       step    optimality
    0          21    6.222482e+00     5.287e+00     1.000e+00     0.000e+00     1.960e+00
```

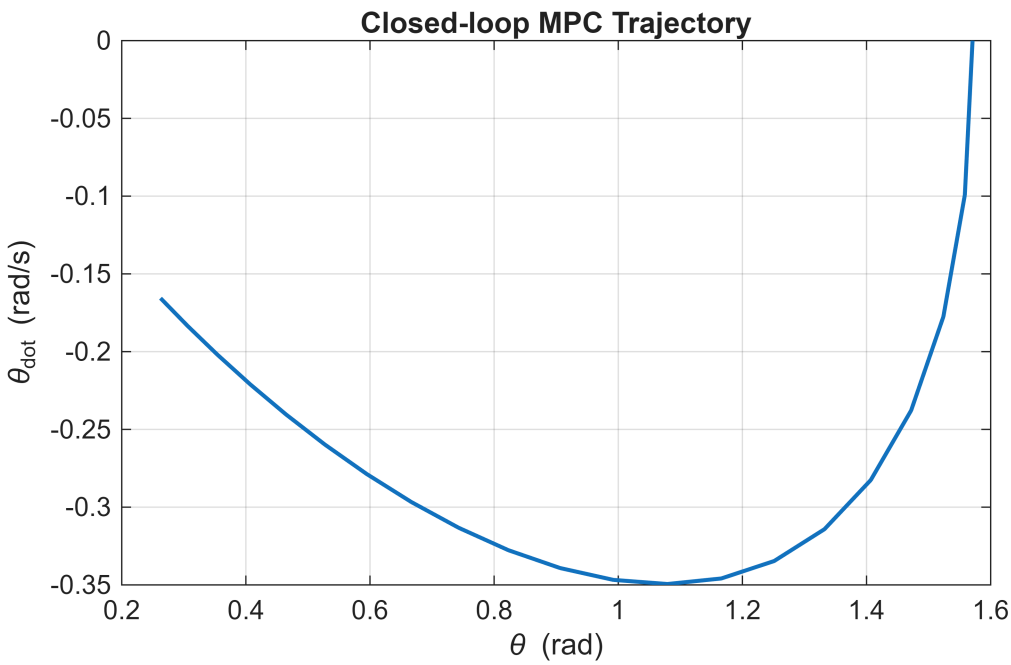
```
    1          42    1.696001e+00     3.886e-16     1.000e+00     3.161e+00     1.421e+00
    2          63    1.482339e-01     4.202e-16     1.000e+00     1.544e+00     3.729e-01
    3          84    7.478815e-02     8.825e-17     1.000e+00     2.710e-01     3.588e-08
    4          85    7.478815e-02     8.825e-17     7.000e-01     5.137e-17     3.132e-08

Local minimum possible. Constraints satisfied.

fmincon stopped because the size of the current step is less than
the value of the step size tolerance and constraints are
satisfied to within the value of the constraint tolerance.

<stopping criteria details>
U_guess = 20×1
    0.3747
    0.1311
    0.4350
    0.0915
    0.6146
    0.0110
    0.5733
    0.7897
    0.2354
    0.4480
    0.5694
    0.0614
    0.4963
    0.6423
    0.2213
      ⋮
 Iter  Func-count            Fval   Feasibility   Step Length       Norm of   First-order
                                                                       step    optimality
    0          21    5.671123e+00     3.633e+00     1.000e+00     0.000e+00     1.942e+00
    1          42    1.250725e+00     5.187e-16     1.000e+00     2.882e+00     1.255e+00
    2          63    1.561786e-01     8.934e-17     1.000e+00     1.386e+00     4.167e-01
    3          84    6.743361e-02     2.333e-16     1.000e+00     2.979e-01     3.830e-08
    4          85    6.743361e-02     2.333e-16     7.000e-01     1.921e-08     3.632e-08

Local minimum possible. Constraints satisfied.

fmincon stopped because the size of the current step is less than
the value of the step size tolerance and constraints are
satisfied to within the value of the constraint tolerance.

<stopping criteria details>
U_guess = 20×1
    0.7466
    0.2369
    0.9573
    0.6203
    0.6003
    0.1726
    0.0903
    0.2553
    0.8586
    0.9111
    0.6996
    0.7252
    0.2299
    0.5761
    0.8106
      ⋮
```

```
 Iter  Func-count            Fval   Feasibility   Step Length       Norm of   First-order
                                                                       step    optimality
    0          21    7.515344e+00     5.544e+00     1.000e+00     0.000e+00     1.977e+00
    1          42    1.990073e+00     3.237e-15     1.000e+00     3.497e+00     1.425e+00
    2          63    1.590446e-01     1.023e-16     1.000e+00     1.703e+00     3.995e-01
    3          84    6.043537e-02     1.496e-16     1.000e+00     3.140e-01     3.770e-08
    4         105    6.043537e-02     4.901e-17     1.000e+00     5.232e-17     3.050e-08

Local minimum possible. Constraints satisfied.

fmincon stopped because the size of the current step is less than
the value of the step size tolerance and constraints are
satisfied to within the value of the constraint tolerance.

<stopping criteria details>
U_guess = 20×1
    0.0606
    0.7257
    0.5566
    0.5294
    0.8300
    0.8588
    0.7890
    0.3178
    0.4522
    0.7522
    0.1099
    0.1097
    0.2699
    0.5246
    0.9727
      ⋮
 Iter  Func-count            Fval   Feasibility   Step Length       Norm of   First-order
                                                                       step    optimality
    0          21    7.599871e+00     5.270e+00     1.000e+00     0.000e+00     1.945e+00
    1          42    2.366107e+00     1.247e-15     1.000e+00     3.685e+00     1.823e+00
    2          63    1.368456e-01     2.984e-16     1.000e+00     1.809e+00     3.586e-01
    3          84    5.375416e-02     2.750e-16     1.000e+00     2.883e-01     4.005e-08
    4         105    5.375416e-02     4.326e-17     1.000e+00     1.954e-16     4.005e-08

Local minimum possible. Constraints satisfied.

fmincon stopped because the size of the current step is less than
the value of the step size tolerance and constraints are
satisfied to within the value of the constraint tolerance.

<stopping criteria details>
U_guess = 20×1
    0.6393
    0.2554
    0.0887
    0.8383
    0.5847
    0.9481
    0.0610
    0.5846
    0.2851
    0.8277
    0.1910
    0.4425
    0.3934
    0.8266
    0.6769
```

```
      ⋮
 Iter  Func-count            Fval   Feasibility   Step Length       Norm of   First-order
                                                                       step    optimality
    0          21    6.004734e+00     4.827e+00     1.000e+00     0.000e+00     1.896e+00
    1          42    1.804315e+00     3.067e-15     1.000e+00     3.236e+00     1.348e+00
    2          63    1.154832e-01     1.934e-16     1.000e+00     1.586e+00     3.384e-01
    3          84    4.740710e-02     1.073e-16     1.000e+00     2.609e-01     4.173e-08
    4          85    4.740710e-02     1.073e-16     7.000e-01     2.722e-08     3.104e-08

Local minimum possible. Constraints satisfied.

fmincon stopped because the size of the current step is less than
the value of the step size tolerance and constraints are
satisfied to within the value of the constraint tolerance.

<stopping criteria details>
U_guess = 20×1
    0.1698
    0.1477
    0.4761
    0.9081
    0.5522
    0.0329
    0.0539
    0.8051
    0.4514
    0.3826
    0.7896
    0.3643
    0.5323
    0.7117
    0.8715
      ⋮
 Iter  Func-count            Fval   Feasibility   Step Length       Norm of   First-order
                                                                       step    optimality
    0          21    6.546016e+00     4.359e+00     1.000e+00     0.000e+00     1.950e+00
    1          42    1.922824e+00     1.665e-16     1.000e+00     3.392e+00     1.356e+00
    2          63    1.236617e-01     4.753e-16     1.000e+00     1.658e+00     4.733e-01
    3          84    4.143515e-02     6.549e-17     1.000e+00     2.868e-01     4.005e-08
    4          85    4.143515e-02     6.549e-17     7.000e-01     2.257e-08     3.032e-08

Local minimum possible. Constraints satisfied.

fmincon stopped because the size of the current step is less than
the value of the step size tolerance and constraints are
satisfied to within the value of the constraint tolerance.

<stopping criteria details>
U_guess = 20×1
    0.4139
    0.3091
    0.2638
    0.7588
    0.9952
    0.1866
    0.7811
    0.1958
    0.9924
    0.8023
    0.4242
    0.7289
    0.4984
```

```
    0.8090
    0.3565
      ⋮

 Iter  Func-count            Fval   Feasibility   Step Length       Norm of   First-order
                                                                       step    optimality
    0          21    7.363086e+00     5.544e+00     1.000e+00     0.000e+00     1.990e+00
    1          42    1.832499e+00     8.778e-16     1.000e+00     3.463e+00     1.401e+00
    2          63    1.487557e-01     2.177e-16     1.000e+00     1.676e+00     4.307e-01
    3          84    3.588288e-02     9.639e-17     1.000e+00     3.360e-01     3.607e-08
    4         105    3.588288e-02     3.716e-17     1.000e+00     3.244e-17     2.912e-08

Local minimum possible. Constraints satisfied.

fmincon stopped because the size of the current step is less than
the value of the step size tolerance and constraints are
satisfied to within the value of the constraint tolerance.

<stopping criteria details>
U_guess = 20×1
    0.7492
    0.0392
    0.9463
    0.7637
    0.5588
    0.1838
    0.4979
    0.5178
    0.9942
    0.8549
    0.9624
    0.6789
    0.4035
    0.9350
    0.4795
      ⋮
```

```
figure;
plot(x_cl(1,:), x_cl(2,:), 'LineWidth', 1.5);
xlabel('\theta (rad)'); ylabel('\theta_d_o_t (rad/s)');
title('Closed-loop MPC Trajectory');
grid on;
```



Problem 1.4: LQ-MPC Solution using State Transformation Approach (KKT Necessary Conditions)

```
% For LQN (linear quadratic finite horizon) with only terminal state
% constraints (no other constraints on input or state vector, apart from
% the forward euler dynamics)

% First define the hamiltonian
syms x_k Q u_k R H  P_k P_k_plus_1 f_k x_k_plus_1 A_k B_k
Hamiltonian = H == 0.5*(transpose(x_k)*Q*x_k + transpose(u_k)*R*u_k) +
transpose(P_k_plus_1*f_k)
```

```
Hamiltonian =
```

$$H = \frac{R\,u_k^2}{2} + \frac{Q\,x_k^2}{2} + P_{k,+1} f_k$$

```
% Necessary Conditions include:
    % Equation of Motion
    Motion = [
    f_k == A_k*x_k + B_k*u_k
    x_k_plus_1 == f_k
    ]
```

```
Motion =
```

$$\begin{pmatrix} f_k = B_k\,u_k + A_k\,x_k \\ x_{k,+1} = f_k \end{pmatrix}$$

```
        % Update H
```

```
        Hamiltonian = subs(Hamiltonian,lhs(Motion), rhs(Motion));
    % Adjoint
    Adjoint = P_k == gradient(rhs(Hamiltonian), x_k)
```

Adjoint = $P_k = Q\,x_k + A_k\,P_{k,+1}$

```
    % Stationarity
    Stationarity = gradient(rhs(Hamiltonian),u_k) == 0
```

Stationarity = $R\,u_k + B_k\,P_{k,+1} = 0$

```
    % Transversality:
```

From cost function $\begin{cases} P = 0 \\ Q = 0 \\ R = I = 1 \end{cases}$ $\quad M = \begin{bmatrix} A \\ A^2 \\ \vdots \\ A^N \end{bmatrix} \quad S = \begin{bmatrix} B & 0 & \cdots & 0 \\ AB & B & \cdots & 0 \\ \vdots & & \ddots & \vdots \\ A^{N-1}B & A^{N-2}B & \cdots & B \end{bmatrix}$

$$P_N = \nabla_{x_N} K + \nabla_{x_N} g(x_N)^T p_N = \cancel{P_N} x_N + M_f^T p_N$$

Terminal cost Matrix, $=0$

Transversality $\longrightarrow$ $\boxed{P_N = \frac{\partial g_N}{\partial x_N} p_N}$

$\downarrow$ $g_N = x_N$

$P_N = I\,p_N$

$\boxed{P_N = p_N}$

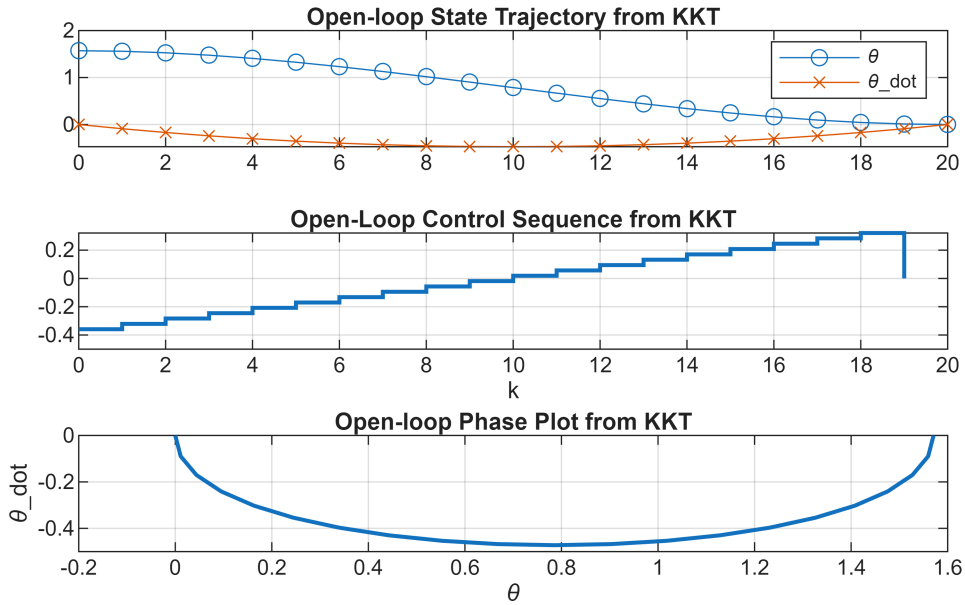
```
% From cost function:
R = 1;
Q = zeros(2);
% Horizon
N=20;

function [u_horizon, x_horizon, p_horizon] = GetKKTHorizon(x0, Ad, Bd, Q, R, N, xN)

    % Evolution:
    phi = [Ad+Bd*inv(R)*transpose(Bd)*inv(Ad')*Q,
-Bd*inv(R)*transpose(Bd)*inv(Ad'); ...
          -transpose(Ad)*Q, inv(transpose(Ad))];
    phiN = phi^N;

    phiN_11 = phiN(1:2,1:2);
    phiN_12 = phiN(1:2,3:4);
    phiN_21 = phiN(3:4,1:2);
    phiN_22 = phiN(3:4,3:4);
```

```
    % Compute p0 from x0
    p0 = (phiN_12)\(xN - phiN_11*x0);

    % Initialize
    x_horizon = [x0,zeros(2,N-1)];
    p_horizon = [p0,zeros(2,N-1)];
    u_horizon = zeros(1,N);

    for i=1:N
    % Starts with x0 and p0
        x_prev = x_horizon(:,i);
        p_prev = p_horizon(:,i);
        xp_prev = [x_prev; p_prev];
    %Find the next augmented xp matrix
        xp_current = phi*xp_prev;
    %Store in x and p horizon vectors
        x_horizon(:,i+1) = xp_current(1:2,:);
        p_horizon(:,i+1) = xp_current(3:4,:);

    %Calculate optimal u_k which leads to x_k+1. No need for u_k at k=N.
        if i ~= N
            u_horizon(:,i) = -R \ Bd'*p_horizon(:,i+1);
        end
    end
end

% Wrap with an anonymous function
GetHorizon = @(x) GetKKTHorizon(x, Ad, Bd, Q, R, N, xN);

[u_horizon, x_horizon, p_horizon] = GetHorizon(x0)
```

```
u_horizon = 1×20
   -0.3590   -0.3212   -0.2835   -0.2457   -0.2079   -0.1701   -0.1323   -0.0945 ···
x_horizon = 2×21
    1.5708    1.5596    1.5271    1.4757    1.4078    1.3257    1.2318    1.1285 ···
         0   -0.0898   -0.1701   -0.2409   -0.3023   -0.3543   -0.3968   -0.4299
p_horizon = 2×21
    0.6047    0.6047    0.6047    0.6047    0.6047    0.6047    0.6047    0.6047 ···
    1.5117    1.3606    1.2094    1.0582    0.9070    0.7559    0.6047    0.4535
```

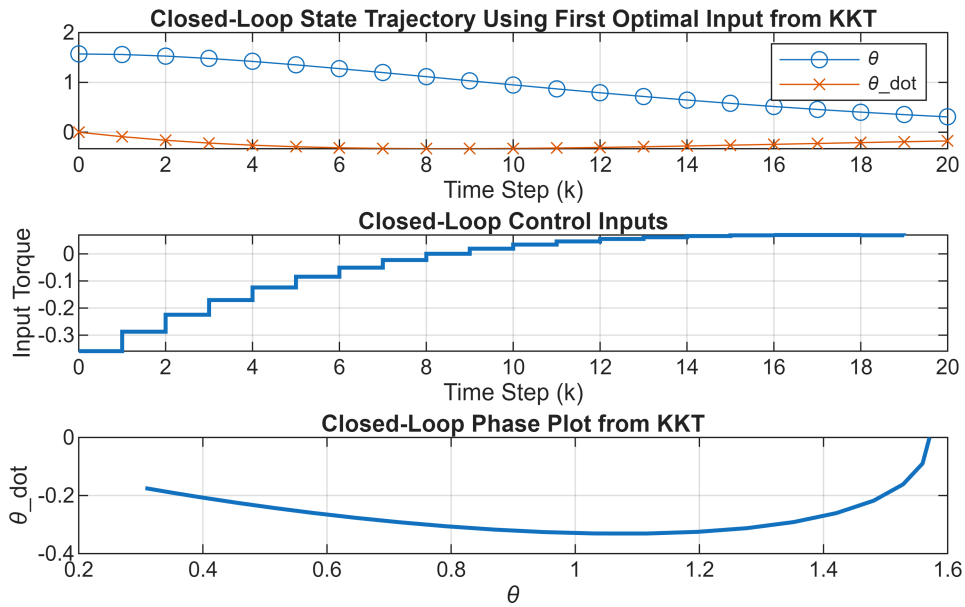
```
% Plot the open loop sequence from KKT solution, horizon N=20, starting @x0
figure;
subplot(3,1,1);
plot(0:N, x_horizon(1,:), '-o'); hold on; plot(0:N, x_horizon(2,:), '-x');
legend('\theta','\theta\_dot'); title('Open-loop State Trajectory from KKT');
grid on;
subplot(3,1,2);
stairs(0:N-1, u_horizon, 'LineWidth', 1.5); title('Open-Loop Control Sequence from
KKT'); xlabel('k'); grid on;
subplot(3,1,3);
```

```
plot(x_horizon(1,:), x_horizon(2,:), 'LineWidth', 1.5); title('Open-loop Phase Plot
from KKT'); xlabel('\theta'); ylabel('\theta\_dot'); grid on;
```




```
% Simulate for 30 time steps, optimizing at each point with KKT function
% above

timesteps = 20;
x_cl2 = [x0, zeros(2,timesteps)];
u_applied2 = zeros(1,timesteps-1);

for i=1:timesteps

    % Get initial State for KKT Optimization. First loop, x_init = x0
    x_init = x_cl2(:,i);
    % Calculate horizon
    [u_horizon, x_horizon, p_horizon] = GetHorizon(x_init);
    % Apply first, find the next system state
    u_applied2(:,i) = u_horizon(:,1);
    x_cl2(:,i+1) = Ad*x_cl2(:,i) + Bd*u_applied2(:,i);

end

figure;
subplot(3,1,1);
plot(0:timesteps, x_cl2(1,:), '-o'); hold on; plot(0:timesteps, x_cl2(2,:), '-x');
xlabel('Time Step (k)');
legend('\theta','\theta\_dot'); title('Closed-Loop State Trajectory Using First
Optimal Input from KKT');
grid on;
subplot(3,1,2);
```

```
stairs(0:timesteps-1, u_applied2, 'LineWidth', 1.5); title('Closed-Loop Control
Inputs'); xlabel('Time Step (k)'); ylabel('Input Torque'); grid on;
subplot(3,1,3);
plot(x_cl2(1,:), x_cl2(2,:), 'LineWidth', 1.5); title('Closed-Loop Phase Plot from
KKT'); xlabel('\theta'); ylabel('\theta\_dot'); grid on;
```



Comparing the results between KKT solution and fmincon: Solutions are very close, see below.

```
ClosedLoopStateDifference = x_cl - x_cl2
```

```
ClosedLoopStateDifference = 2×21
         0   -0.0012   -0.0044   -0.0089   -0.0141   -0.0198   -0.0254   -0.0308 ···
         0   -0.0094   -0.0160   -0.0201   -0.0222   -0.0228   -0.0222   -0.0208
```

```
ClosedLoopInputDifference = u_applied - u_applied2
```

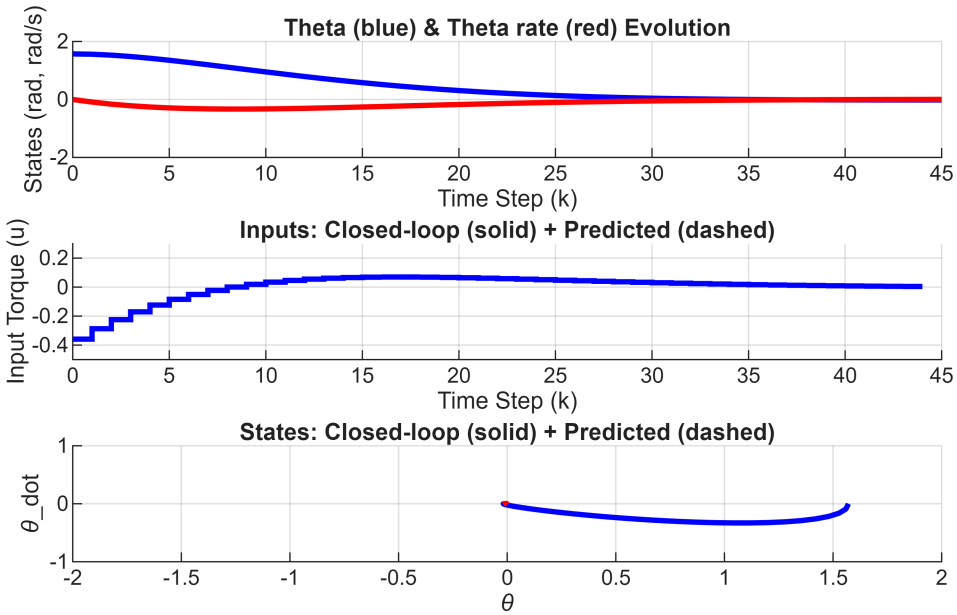
```
ClosedLoopInputDifference = 1×20
   -0.0378   -0.0261   -0.0164   -0.0086   -0.0024    0.0024    0.0059    0.0085 ···
```

```
% ChatGPT used for 'video plots' of the solutions Below:
%% ================================================================
% ANIMATION of KKT receding-horizon execution
%   subplot 1 → (theta,theta_dot) vs k (closed-loop + predicted)
%   subplot 2 → (theta,theta_dot) closed-loop + predicted
%   subplot 3 → u closed-loop + predicted
% ================================================================

timesteps = 45;     % animate 35 time steps
Ts_plot = 0.25;     % seconds between frames

x_cl3 = zeros(2,timesteps+1);
x_cl3(:,1) = x0;
```

```
u_applied3 = zeros(1,timesteps);

figure; clf;

% ------- STATE DEVELOPMENT subplot (top) -------
ax0 = subplot(3,1,1); hold on; grid on;
title(ax0,'Theta (blue) & Theta rate (red) Evolution');
xlabel(ax0,'Time Step (k)');
ylabel('States (rad, rad/s)')

% predicted state handle
pred_x_k_line    = plot(ax0,NaN,NaN,'b--','LineWidth',1.5);
pred_xdot_k_line = plot(ax0,NaN,NaN,'r--','LineWidth',1.5);

% ------- INPUT subplot (middle) -------
ax2 = subplot(3,1,2); hold on; grid on;
title(ax2,'Inputs: Closed-loop (solid) + Predicted (dashed)');
xlabel(ax2,'Time Step (k)'); ylabel(ax2,'Input Torque (u)');
cl_input_line  = stairs(ax2,NaN,NaN,'b','LineWidth',2);
pred_input_line = stairs(ax2,NaN,NaN,'r--','LineWidth',2);

% ------- STATE subplot (bottom) -------
ax1 = subplot(3,1,3); hold on; grid on;
title(ax1,'States: Closed-loop (solid) + Predicted (dashed)');
xlabel(ax1,'\theta'); ylabel(ax1,'\theta\_dot');
cl_state_line = plot(ax1,NaN,NaN,'b','LineWidth',2);
pred_state_line = plot(ax1,NaN,NaN,'r--','LineWidth',2);
% closed-loop handles
cl_x_k_line    = plot(ax0,NaN,NaN,'b','LineWidth',2);   % theta
cl_xdot_k_line = plot(ax0,NaN,NaN,'r','LineWidth',2);   % theta_dot

for k = 1:timesteps

    % ============================================================
    % 1) CLOSED-LOOP PROPAGATION
    % ============================================================
    x_init = x_cl3(:,k);

    % horizon prediction from KKT
    [u_horizon, x_horizon, p_horizon] = GetHorizon(x_init);

    % store applied u
    u_applied3(k) = u_horizon(1);

    % propagate system with first horizon input
    x_cl3(:,k+1) = Ad*x_init + Bd*u_applied3(k);

    % ============================================================
    % 2) UPDATE TOP SUBPLOT — STATE vs k
    % ============================================================
```

```
% closed-loop trajectory so far
set(cl_x_k_line,   'XData',0:k, ...
                   'YData',x_cl3(1,1:k+1));   % theta

set(cl_xdot_k_line,'XData',0:k, ...
                   'YData',x_cl3(2,1:k+1));   % theta_dot

% predicted horizon from current step
k_pred = k : k+size(x_horizon,2)-1;
set(pred_x_k_line,   'XData',k_pred, 'YData',x_horizon(1,:));   % theta
set(pred_xdot_k_line,'XData',k_pred, 'YData',x_horizon(2,:));   % theta_dot

xlim(ax0,[0 timesteps]);
ylim(ax0,[-2 2]);


% ============================================================
% 3) UPDATE MIDDLE SUBPLOT — INPUT vs k
% ============================================================

% closed-loop inputs so far
set(cl_input_line, 'XData',0:k-1, ...
                   'YData',u_applied3(1:k));

% predicted future inputs
set(pred_input_line,'XData',k:k+N-1, ...
                    'YData',u_horizon);

xlim(ax2,[0 timesteps]);
ylim(ax2,[-0.5 0.3]);


% ============================================================
% 4) UPDATE BOTTOM SUBPLOT — PHASE PLOT (θ vs θ̇)
% ============================================================

% closed-loop phase trajectory
set(cl_state_line,'XData',x_cl3(1,1:k+1), ...
                  'YData',x_cl3(2,1:k+1));

% predicted horizon phase trajectory
set(pred_state_line,'XData',x_horizon(1,:), ...
                    'YData',x_horizon(2,:));

xlim(ax1,[-2 2]);
ylim(ax1,[-1 1]);


% ============================================================
```

```
    % 5) DRAW FRAME
    % ============================================================
    drawnow;
    pause(Ts_plot);

end
```



Problem 1.5: P-LQN Formulation & solution with quadprog.m

There is no terminal constraint, and there is no state cost, so the optimal control input is **zero**. The controller has no requirement to balance the pendulum, and zero input over the time horizon minimizes the cost function.

$$\min_{U \in \mathbb{R}^{N n_u}} \quad \mathcal{J}_N = \frac{1}{2}\left(U^T H U + 2q^T U\right)$$
$$\text{subject to} \quad GU \le \tilde{W}$$

MATLAB

where $\tilde{W} = W + Tx_0$, $H = S^T \bar{Q} S + \bar{R}$, and $q = S^T \bar{Q} M x_0$.

quadprog(H,q,G,$\tilde{W}$)

solveQP(H, q, G, $\tilde{W}$, options)

Common optional parameters (options) are

- Bounds for the variables (min and max)
- Initial solution guess (otherwise the solver finds one)
- Maximum number of iteration (or max time)
- Termination condition $(\epsilon, ||\mathcal{J}_N(U) - \mathcal{J}_N(U^*)|| \le \epsilon)$
- Constraint acceptance $(\epsilon, ||GU - \tilde{W}|| \le \mathbf{1}\epsilon)$

"gradient projection algo"

tolerance for optimality

relaxing constraint by tolerance ϵ can give same improvement (given by magnitude of input Lagrange multiplier) or find quicker soln.

In MATLAB:

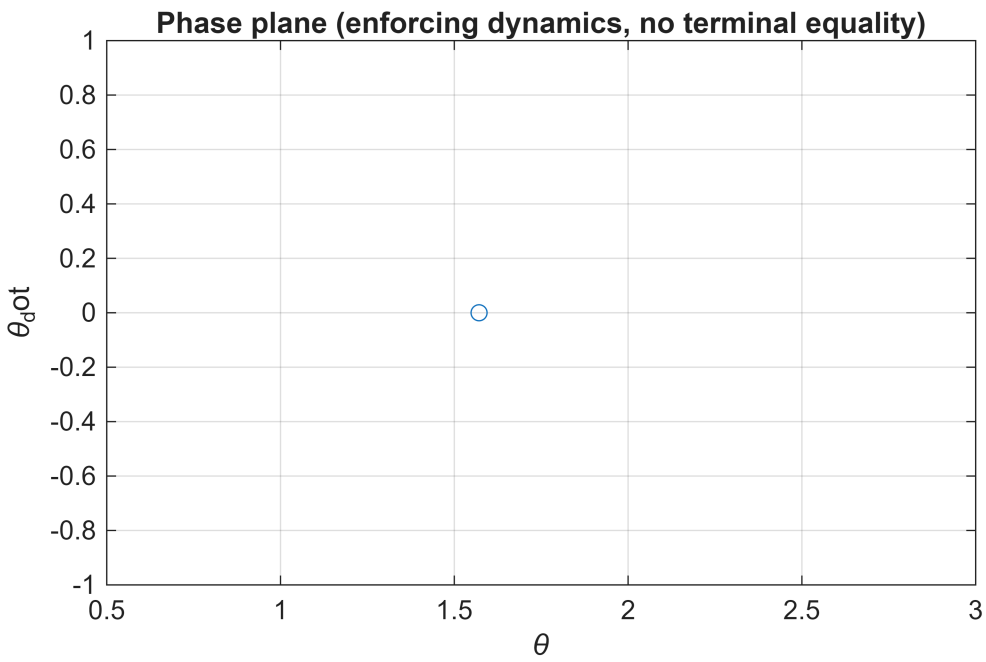
quadprog(H, f, A, B)

$\min \frac{1}{2} x^T H x + f^T x$

s.t. $Ax \le B$

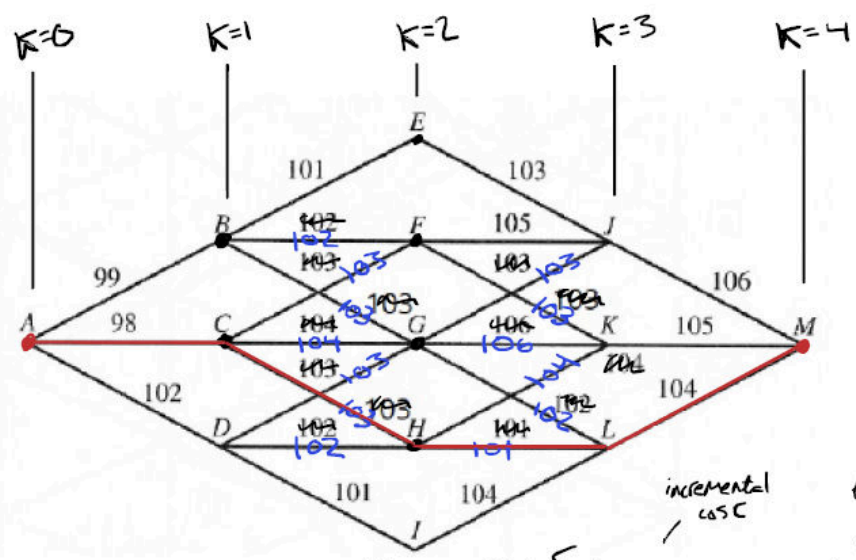
```
% % ChatGPT was used for this section of the script

% --- parameters (edit if needed) ---
Ts = 0.25;
N  = 20;            % horizon
nx = 2;             % state dimension
nu = 1;             % input dimension

% discrete system (your values)
A = [1 Ts; 0 1];
B = [0.5*Ts^2; Ts];

% initial condition x0 (example: pi/2, 0)
x0 = [pi/2; 0];

% --- decision variable sizes ---
nXvars = nx * N;          % x1..xN
nUvars = nu * N;          % u0..u_{N-1}
nVars  = nXvars + nUvars;

% --- cost: J = sum_{k=0}^{N-1} u_k^2 ---
H = zeros(nVars);
% put identity on the u-block:
u_idx = nXvars + (1:nUvars);      % column indices for u in z
H(u_idx,u_idx) = eye(nUvars);     % weight = 1 on each u_k
f = zeros(nVars,1);

% --- equality constraints: enforce dynamics x_{k+1} - A x_k - B u_k = 0 ---
Aeq = zeros(nx*N, nVars);
beq = zeros(nx*N, 1);
```

```
for k = 0:(N-1)
    row_idx = k*nx + (1:nx);         % rows for this dynamic equation (for x_{k+1})
    % column indices for x_{k+1} variable:
    col_xkp1 = (k)*nx + (1:nx);      % because x1 starts at index 1 when k=0

    % place +I on x_{k+1}
    Aeq(row_idx, col_xkp1) = eye(nx);

    % place -A on x_k (except when k==0; x0 is known)
    if k >= 1
        col_xk = (k-1)*nx + (1:nx);  % x_k variable column indices
        Aeq(row_idx, col_xk) = -A;
    end

    % place -B on u_k column
    col_uk = nXvars + k*nu + (1:nu);   % u_k column indices
    Aeq(row_idx, col_uk) = -B;

    % RHS: for k==0 move A*x0 to beq
    if k == 0
        beq(row_idx) = A * x0;
    else
        beq(row_idx) = zeros(nx,1);
    end
end

% no inequalities or bounds in this example
Aineq = [];
bineq = [];
lb = [];
ub = [];

% --- solve QP ---
opts = optimoptions('quadprog','Display','off');
z_opt = quadprog(H,f,Aineq,bineq,Aeq,beq,lb,ub,[],opts);

% --- parse result ---
X_opt = reshape(z_opt(1:nXvars), nx, N);    % columns are x1..xN
U_opt = z_opt(nXvars+1:end);                % u0..u_{N-1}

% include x0 to plot full state trajectory
X_full = [x0, X_opt];   % columns x0..xN

% quick plot (theta vs theta_dot phase plane)
figure;
plot(X_full(1,:), X_full(2,:), '-o');
xlabel('\theta'); ylabel('\theta_dot');
title('Phase plane (enforcing dynamics, no terminal equality)');
grid on;
```



ECE675 HW3 Problem 2:

ELE675
HW3
Problem 2
Dynam. Prog.


K=0
K=1
K=2
K=3
K=4
A
B
C
D
E
F
G
H
I
J
K
L
M
99
98
102
101
103
105
106
105
104
101
104
102
103
103
103
103
103
103
103
103
103
103
103
102
102
101
101
102
103
103
103
103
103
103
103
103
103
103
103

ECE675 HW3 Problem 3:

Problem 3 The inverted pendulum system consists of an inverted pendulum mounted on a cart shown in Fig.2.

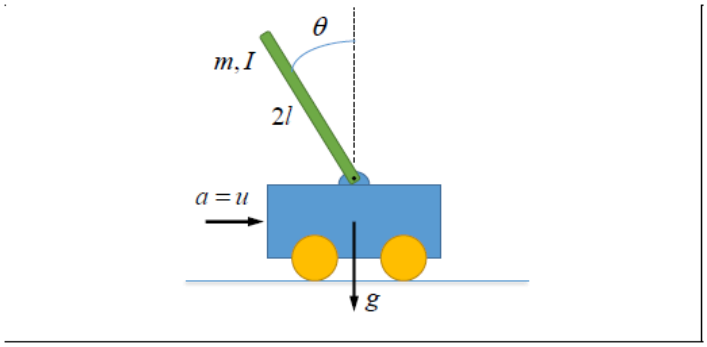


Figure 2: Inverted pendulum on a cart

The dynamics of the inverted pendulum are modeled in state-space form as follows:

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} x_2 \\ -a_0 \sin x_1 - a_1 x_2 \end{bmatrix} + \begin{bmatrix} 0 \\ b_0 \cos x_1 u \end{bmatrix}$$

where $x = [x_1\, x_2]^T = [\theta\, \dot{\theta}]^T$, and $a_0 = -\frac{mgl}{I+ml^2}$, $a_1 = \frac{b}{I+ml^2}$, and $b_0 = \frac{ml}{I+ml^2}$. For the considered system, the parameters are: $a_0 = -2.5$, $a_1 = 0.05$, and $b_0 = 0.25$.

(1) Determine the linearized dynamics about the equilibrium $x = [0\, 0]^T$.

(2) Convert the continuous-time dynamics to discrete-time using $\Delta t = 0.1$.

(3) Design a feedback control law $u = -Kx$ using Matlab command `dlqr` for the discretized system in (2). Set $Q = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$ and $R = 1$. Simulate the inverted pendulum system in closed-loop with the designed controller and plot the results of the simulation.

```
x0 = [pi/4; 0];
sim('Prob3a.mdl',[0 10])

figure
subplot(2,1,1)
plot(x.time,x.signals.values)
ylim([-pi/2 pi/2])
legend('\theta','d\theta/dt')
grid on
subplot(2,1,2)
plot(u.time,u.signals.values)
ylim([-pi pi])
grid on
```

Problem 3.1: Linearized Dynamics

```
syms a0 a1 b0 x1 x2

% Linearize at x = [0; 0], cos(x1) --> 1, sin(x1) --> x1
A_lin = [0, 1; -a0, -a1]
```

A_lin =

$$\begin{pmatrix} 0 & 1 \\ -a_0 & -a_1 \end{pmatrix}$$

```
B_lin = [0; b0]
```

B_lin =

$$\begin{pmatrix} 0 \\ b_0 \end{pmatrix}$$

```
% Substitute in the numerical given values:
A_lin = double(subs(A_lin,[a0,a1,b0],[-2.5,0.05,0.25]))
```

```
A_lin = 2×2
         0    1.0000
    2.5000   -0.0500
```

```
B_lin = double(subs(B_lin,[a0,a1,b0],[-2.5,0.05,0.25]))
```

```
B_lin = 2×1
         0
    0.2500
```

Problem 3.2: Discretize the Dynamics:

```
% Sample Rate
ts = 0.1;
% Intitial Cond.
x0 = [pi/4; 0];

% Define continuous system, build discrete ss system
ContLinSystem = ss(A_lin,B_lin,zeros(2,2),zeros(2,1))
```

```
ContLinSystem =

  A =
           x1     x2
   x1       0      1
   x2     2.5  -0.05

  B =
         u1
   x1     0
   x2  0.25

  C =
       x1  x2
   y1   0   0
   y2   0   0

  D =
       u1
   y1   0
   y2   0

Continuous-time state-space model.
Model Properties
```

```
DiscLinSystem = c2d(ContLinSystem,ts);

% Extract A & B for discrete system
Ad_lin = DiscLinSystem.A
```

```
Ad_lin = 2×2
```

```
    1.0125    0.1002
    0.2504    1.0075
```

```
Bd_lin = DiscLinSystem.B
```

```
Bd_lin = 2×1
    0.0013
    0.0250
```

```
Cd_lin = eye(2,2)
```

```
Cd_lin = 2×2
     1     0
     0     1
```

```
Dd_lin = DiscLinSystem.D
```

```
Dd_lin = 2×1
     0
     0
```

Problem 3.3: Discrete LQR Controller

```
Q = [1 0; 0 0];
R = 1;

[K_dlqr, S, P] = dlqr(Ad_lin,Bd_lin,Q,R)
```

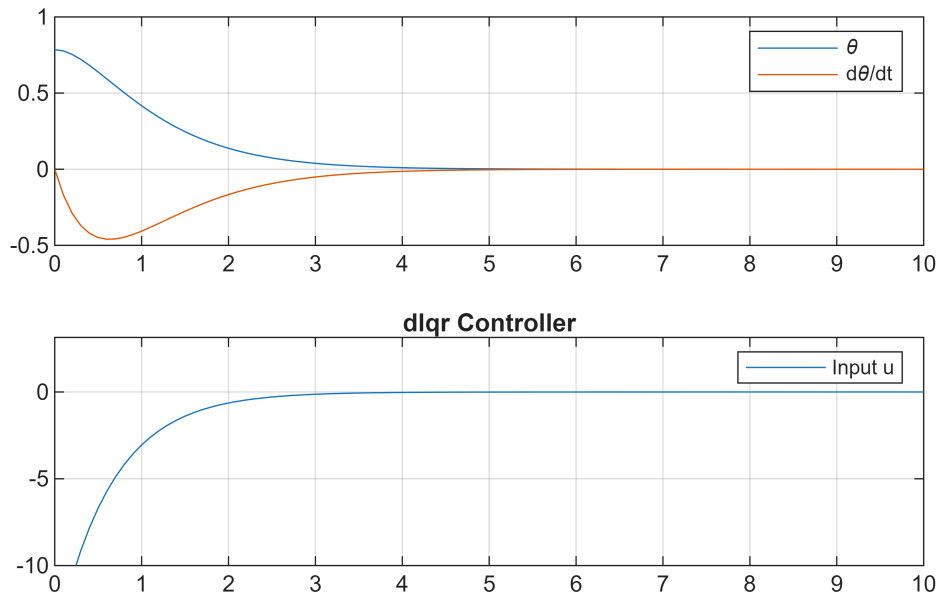
```
K_dlqr = 1×2
   18.5997   11.5669
S = 2×2
10^3 ×
    1.2961    0.8036
    0.8036    0.4997
P = 2×1 complex
   0.8535 + 0.0064i
   0.8535 - 0.0064i
```

```
simout1 = sim('ECE675_HW3_Problem3_sim.mdl',[0 10])
```

```
simout1 
  Simulink.SimulationOutput:

                logsout: [1x1 Simulink.SimulationData.Dataset]
                   tout: [101x1 double]

     SimulationMetadata: [1x1 Simulink.SimulationMetadata]
           ErrorMessage: [0x0 char]
```

```
theta = simout1.logsout{1}.Values.Data(:,1);
theta_dot = simout1.logsout{1}.Values.Data(:,2);
time = simout1.logsout{1}.Values.Time(:,1);
input = simout1.logsout{2}.Values.Data(:,1);

figure
subplot(2,1,1)
```

```
plot(time,theta)
hold on
plot(time, theta_dot)
legend('\theta','d\theta/dt')
grid on
subplot(2,1,2)
plot(time, input)
ylim([-10,pi])
legend('Input u')
title('dlqr Controller')
grid on
```

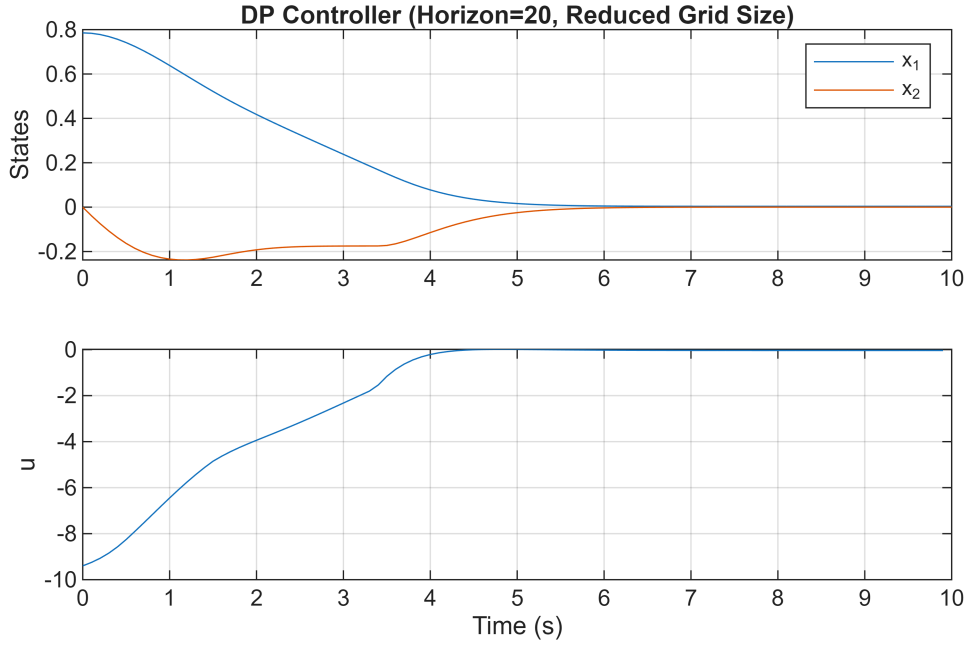


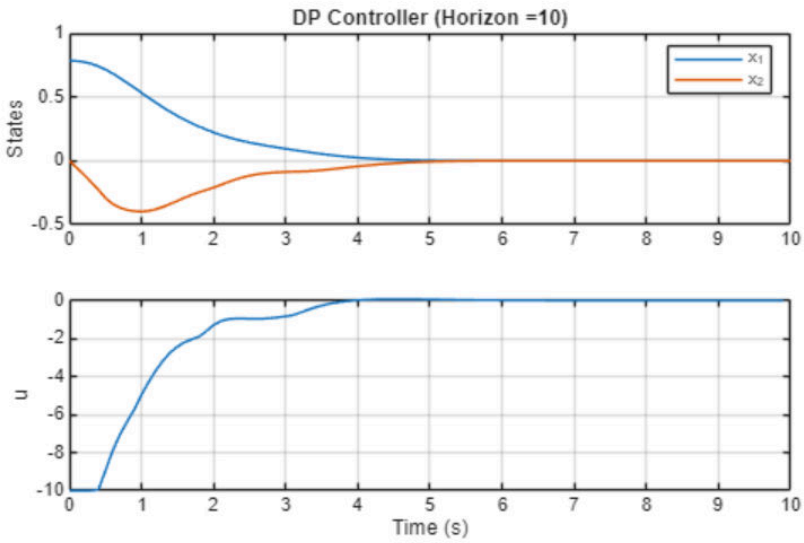
Problem 3.4: Dynamic Program Control Law

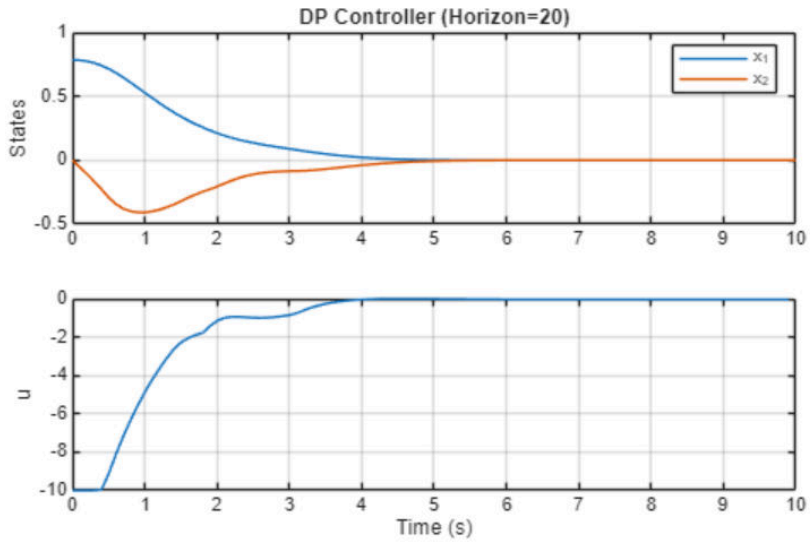
```
% Get discrete LQR terminal cost matrix using dlqr (you already computed)
% [K_dlqr, S, ~] = dlqr(Ad_lin,Bd_lin,Q,R);   % S is solution of discrete riccati
(use your existing S)

% params
X1GridSize = 10;
X2GridSize = X1GridSize;
DP_horizon = 20;
Q = [1 0; 0 0];
R = 1;
ts = 0.1;

% stage + terminal cost
Jstage = @(x,u) x'*Q*x + u'*R*u;
```

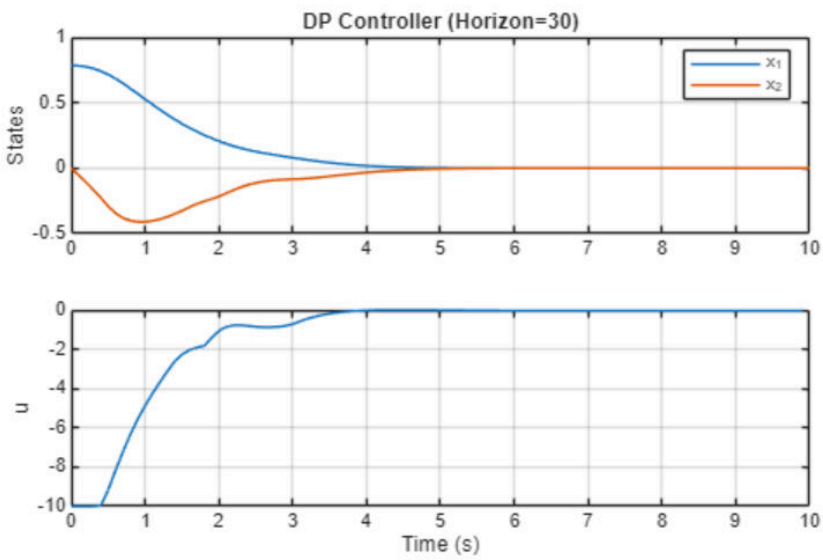
```
Jtogo = @(x) x'*S*x;

% state grid
X1 = linspace(-pi/2, pi/2, X1GridSize);
X2 = linspace(-pi/2, pi/2, X2GridSize);

% initialize tables for cost & optimal inputs
U = zeros(X1GridSize, X2GridSize);
J  = zeros(X1GridSize, X2GridSize);

% bounded u search grid
umax = 10; % control bounds should be within this pretty easily
u_grid = linspace(-umax, umax, 201);

% Value iteration
for iter = 1:DP_horizon
    for i = 1:length(X1)
        for j = 1:length(X2)
            x = [X1(i); X2(j)];

            % evaluate cost for every u candidate
            % next state
            Xnext = Ad_lin*x + Bd_lin * u_grid;   % 2 x numU
            % compute cost: stage + terminal-on-next
            cost_vals = zeros(size(u_grid));
            for m = 1:numel(u_grid)
                xn = Xnext(:,m);
                % evaluate Jtogo at xn via interp2 on J table (if iter>1)
                if iter==1
                    future_cost = Jtogo(xn);  % currently x'*S*x
                else
                    % safe interp: clamp xn to grid before interp
                    x1c = min(max(xn(1), min(X1)), max(X1));
                    x2c = min(max(xn(2), min(X2)), max(X2));
                    future_cost = interp2(X2, X1, J, x2c, x1c, 'linear');
                end
                cost_vals(m) = Jstage(x, u_grid(m)) + future_cost;
            end

            % pick best u
            [mincost, midx] = min(cost_vals);
            U(i,j) = u_grid(midx);
            J(i,j) = mincost;
        end
    end

    % update Jtogo to the just-computed J table
    % NOTE: J is arranged as J(i over X1, j over X2)
    Jtogo = @(xq) interp2(X2, X1, J, ...
                          min(max(xq(2), min(X2)), max(X2)), ...
```

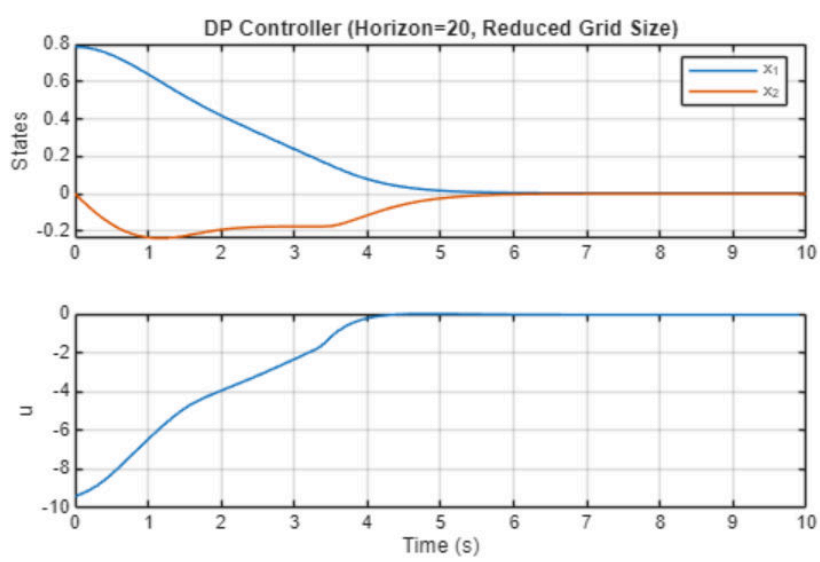
```
                            min(max(xq(1), min(X1)), max(X1)), 'linear');
end

% DP policy accessor (with clamping)
function u = GetInput_DP(x_state, X1, X2, U)
    x1 = x_state(1);
    x2 = x_state(2);
    % clamp
    x1c = min(max(x1, min(X1)), max(X1));
    x2c = min(max(x2, min(X2)), max(X2));
    u = interp2(X2, X1, U, x2c, x1c, 'linear');
end

GetOptimalInput = @(x_state) GetInput_DP(x_state, X1, X2, U);

%% === Simulate closed-loop with DP policy ===
Tfinal = 10;
timesteps = round(Tfinal/ts);

x_cl_DP = zeros(2, timesteps+1);
u_cl_DP = zeros(1, timesteps);

x_cl_DP(:,1) = x0;

for k = 1:timesteps
    curx = x_cl_DP(:,k);
    u = GetOptimalInput(curx);
    u_cl_DP(k) = u;
    x_cl_DP(:,k+1) = Ad_lin*curx + Bd_lin*u;
end

t = 0:ts:Tfinal;

% plots
figure;

subplot(2,1,1)
plot(t, x_cl_DP(1,:), t, x_cl_DP(2,:))
grid on
legend('x_1','x_2')
title('DP Controller (Horizon=20, Reduced Grid Size)')
ylabel('States')

subplot(2,1,2)
plot(t(1:end-1), u_cl_DP)
grid on
ylabel('u')
xlabel('Time (s)')
```


DP Controller (Horizon=20, Reduced Grid Size)
States
x_1
x_2
u
Time (s)


DP Controller (Horizon =10)
States
x_1
x_2
u
Time (s)


DP Controller (Horizon=20)
States
u
Time (s)
x1
x2


DP Controller (Horizon=30)
States
u
Time (s)
x1
x2


DP Controller (Horizon=20, Reduced Grid Size)
States
u
Time (s)
x1
x2

(5) Design an MPC-based feedback control law using a Matlab **quadprog** and compare the results with those in (4).

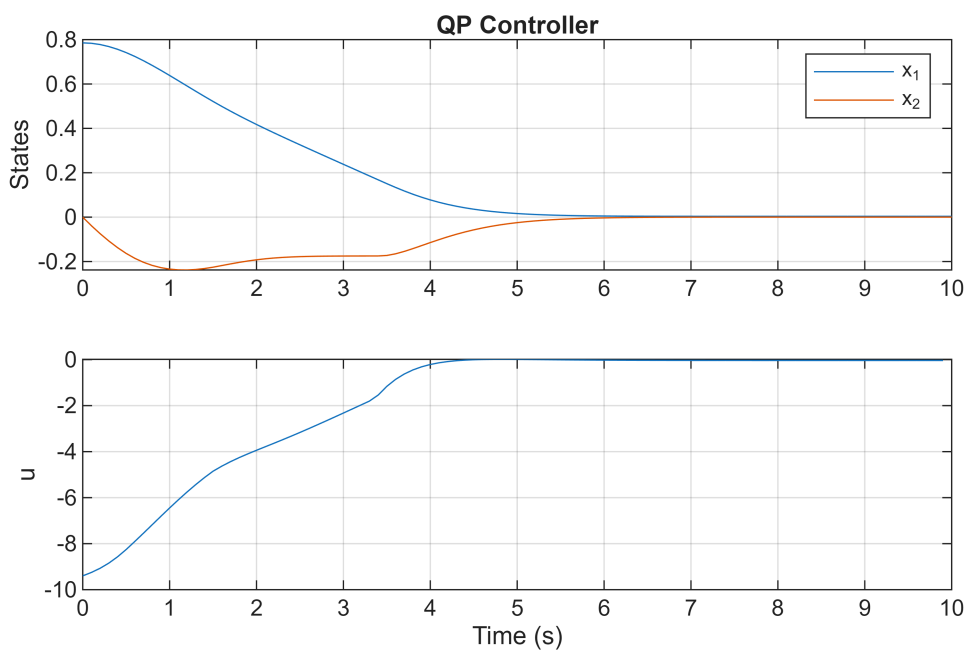
Closed loop response with quadprog is very similar to response using the DP control law with reduced grid size (10x10 grid). Increased grid size on the DP controller significantly reduces the control effort to bring the pendulum to upright position.

```
function u_opt = GetOptimalInputQP(x, Ad, Bd, Q, R, P)

% state-dependent
Ax = Ad * x;

% quad term
H = Bd' * P * Bd + R;
H = (H + H')/2;

% linear term
f = (Bd' * P * Ax);

% solve QP
u_opt = quadprog(H, f, [], [], [], [], [], [], [], ...
                 optimoptions('quadprog','Display','off'));

end

DP_horizon = 20;
Q = [1 0; 0 0];
R = 1;
P = zeros(2,2)
```

```
P = 2×2
     0     0
     0     0
```

```
ts = 0.1;

GetOptimalInputQPAnon = @(x) GetOptimalInputQP(x, Ad, Bd, Q, R, P)
```

```
GetOptimalInputQPAnon = function_handle with value:
    @(x)GetOptimalInputQP(x,Ad,Bd,Q,R,P)
```

```
Tfinal = 10;
timesteps = round(Tfinal/ts);

x_cl_QP = zeros(2, timesteps+1);
u_cl_QP = zeros(1, timesteps);

x_cl_QP(:,1) = x0;
```

```
for k = 1:timesteps
    curx = x_cl_QP(:,k);
    u = GetOptimalInput(curx);
    u_cl_QP(k) = u;
    x_cl_QP(:,k+1) = Ad_lin*curx + Bd_lin*u;
end

t = 0:ts:Tfinal;

% plots
figure;

subplot(2,1,1)
plot(t, x_cl_QP(1,:), t, x_cl_QP(2,:))
grid on
legend('x_1','x_2')
title('QP Controller')
ylabel('States')

subplot(2,1,2)
plot(t(1:end-1), u_cl_QP)
grid on
ylabel('u')
xlabel('Time (s)')
```